

Agent RL do uproszczonego Clash Royale

Raport z projektu
Autor: Piotr Jakubczyk
Data: 16.06.2026

1. Cel projektu

Celem projektu było zaprojektowanie i implementacja uproszczonej symulacji gry inspirowanej Clash Royale oraz wytrenowanie agenta uczenia ze wzmocnieniem (RL), który podejmuje skuteczne decyzje taktyczne w czasie rzeczywistym — wybór karty, strefy zagrania i momentu ataku/obrony.

Problem: jak nauczyć komputer grać w grę strategiczną bez podłączania się do prawdziwego Clash Royale. Wymaga to kontrolowanego środowiska, w którym da się zdefiniować stan gry, przestrzeń akcji, nagrodę i przeciwnika treningowego.

- Cele szczegółowe:
 - Zbudować symulację areny z eliksirem, kartami i walką jednostek.
 - Zaimplementować bota regułowego jako benchmark.
 - Wytrenować agenta MaskablePPO i porównać go z baseline.

2. Jak rozwiązano problem

2.1 Środowisko symulacji

Rdzeń to moduł `board.py`: arena 18×32 , dwa mosty, 6 kart w talii (Knight, Giant, Cannon, Musketeer, Hog_Rider, Fireball), eliksir, wieże i walka jednostek. Wrapper `gym_env.py` udostępnia API Gymnasium — gracz 0 (RL) vs gracz 1 (LogicAgent).

2.2 Reprezentacja stanu i akcji

Sekcja	Indeksy	Opis
Globalne	0–2	Eliksir $\times 2$ (swoj i przeciwnika), czas meczu
Wieże	3–8	HP 6 wież
Ręka	9–32	One-hot 4 slotów $\times$ 6 kart
Jednostki	33–152	24 jednostki $\times$ 5 cech

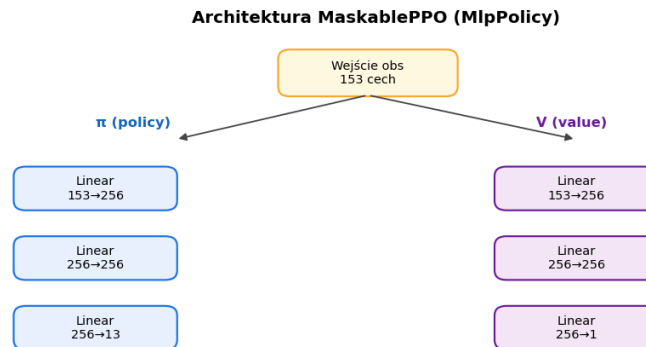
Przestrzeń akcji: 13 możliwości (0 = pass, 1–12 = slot karty $\times$ strefa deploy). Nagroda: zmiana łącznego HP wież wroga minus własnych. MaskablePPO odcina nielegalne akcje (brak eliksiru na kartę).

2.3 Bot regułowy (LogicAgent)

Przeciwnik treningowy oparty na regułach: snapshot planszy → generowanie kandydatów zagrania → scoring → wybór najlepszego. Obsługuje obronę (Hog, Giant), Fireball, push i cycle eliksiru. Stanowi trudny, powtarzalny benchmark.

2.4 Agent RL i trening

Algorytm: MaskablePPO (Stable-Baselines3), sieć MLP z osobnymi gałęziami policy [256,256] i value [256,256]. Trening: 1 mln kroków, 8 środowisk równoległych, batch 256, entropi 0,02. Logi: TensorBoard, checkpointy co 100k.



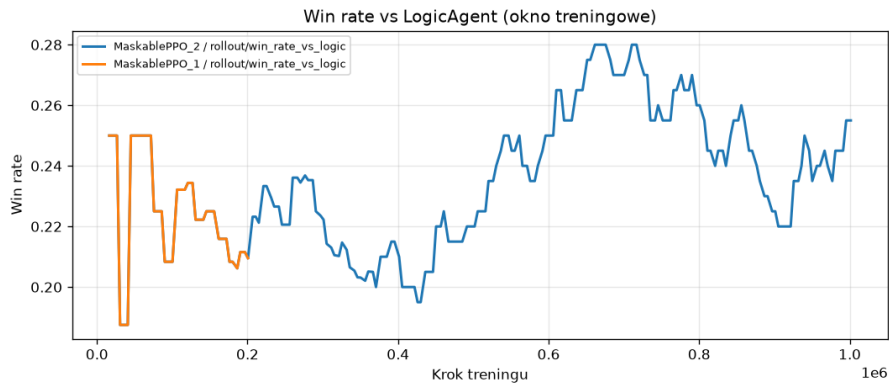
Rys. 1. Architektura sieci (wejście 153 → akcja 13 / wartość 1)

3. Wyniki

Ewaluacja w tym samym układzie co trening: agent na P0 vs LogicAgent na P1 (gym_env).

Agent	Epizody	Wygrane	Win rate
Losowy (bez modelu)	30	14	46,7%
ppo_cr_final (po treningu)	100	69	69,0%
Logic vs Random (osobny test)	50	50 Logic	100% Logic

Losowy baseline (46,7%) to RLAgent bez wczytanego modelu — losowy wybór legalnych akcji. Po treningu ppo_cr_final osiąga 69%, co potwierdza, że sieć nauczyła się strategii lepszej niż przypadkowe zagrania. Wzrost: ok. +22 p.p. win rate.



Rys. 2. Win rate w trakcie treningu

4. Wnioski

1. Cel osiągnięty: agent RL wygrywa z botem regułowym znacznie częściej niż losowy gracz.
2. Kluczowe elementy rozwiązania: jawna symulacja, maska akcji, nagroda z HP wież, LogicAgent jako przeciwnik.
3. Prosta sieć MLP [256,256] wystarcza przy 153 cechach wejściowych.
4. Ograniczenia: uproszczona gra (6 kart), brak self-play, trening vs jeden typ przeciwnika.